

Implementing Early Exits on AI Models

Amirreza Tarabkhah

amirreza.tarabkhah@stud.uni-heidelberg.de

Abstract

One of the challenges from the beginning of AI usage is making the models more efficient. It can be implemented in the cost efficient or time efficient way. Using optimizers, quantize the models, pruning or even using stronger hardware are some ways to overcome these problems. Although they are always efficient, there is a problem that they didn't solve yet. Most of the data are easy to predict and there is no need to kill memory, time and other sources to predict them. Stronger models can predict really hard-to-predict data but they use a big bunch of source to predict a data. In this project, I will overcome this problem by using output layers before the main output and they are named Early-Exit (EE) layers. By using this idea, the data that are easy-to-predict, will exit early from the AI model earlier so there is no need to waste the source to predict them more.

1 Introduction

Implementing AI models to predict the data based on the tasks is the most trend happening in the world nowadays. From detecting apple or orange to text generation and making image are done by AI models. Due to the heavy hardware and time usage of these models, one of the every time challenges is optimizing them or making them more efficient. For instance, by quantizing the model, we can use more weaker hardware to run the models. The other example is related to inference time. Two parameters are important during inference, time and accuracy. Some models are consuming more time to give more accurate results and the others sacrifice the accuracy to consume less time. If you use weak model, the accuracy would be less than strong model but the time consumption is also less. You cannot run a weak model at first but then when you face a strong data, terminate the model and run the other strong one. So, what will happen if there is a model that can handle both of them?

There is a one way to implement more than one output layer for an AI model and run that. This scenario has two advantages. First, although you use strong models, when you input a easy-to-predict data, you can exit the data as soon as the model predict it. Second, you can mix two models together. Feeding some simple data like MNIST to a strong model like ResNet families, is killing the source. Some really simple models like LeNet can also predict MNIST data with much less source. The first two elements in both model look like each other. Therefore, it is possible to make a output line that makes the first exit way something like LeNet inside ResNet.

To implement this project, I used ResNet-18 and ResNet-24 with zero, three, five EEs and in all of them, the first exit is a LeNet shaped model. A simple model that is not really deep and really shallow and perfect for detecting easy-to-predict data. Also, I will describe a trade-off between accuracy and speed using threshold τ .

Finally, after implementing a structure like that, there is a possibility of making inference time less than a half. Of course the accuracy will decrease, but it will decrease less than 15 percent.

2 Dataset

The experiments utilize the "Cifar-10" dataset for implementation. Size of each image in dataset is 32 by 32. Here, it has been resized to 224 by 224. Also, other details like horizontal flip, rotation and crop are implemented for some random selection of data. Detailed statistics for the train dataset is presented in Table 1.

3 Algorithm

To implement the model, the ModuleList from PyTorch should be used. It is mandatory due to implementation of Early-Exits. Most important part of

Split	#sentences
Training	50000
Validation	10000
Testing	10000

Table 1: Counts of the different data splits for Cifar-10

the ResNet is shortcut connections. By using the shortcut connections, it is possible to program and train really deer models even for small amount of data. Therefore, it is important to keep the effect of shortcut connections. Because of that, I named each blocks connected with shortcut connection as residual block.

There is two types of blocks. (I) Normal residual block and (II) Bottleneck residual block from (He et al., 2015):

1. Basic Residual Block (ResidualBlock):

This block follows the standard ResNet-18/34 design: two stacked 3×3 convolutions (each followed by BatchNorm and ReLU), with the identity shortcut added before the final ReLU. When the input and output channel counts differ, a 1×1 convolution projects the shortcut to match dimensions (i.e., a projection shortcut instead of a pure identity).

2. Bottleneck Residual Block (BottleneckResidualBlock):

This block follows the ResNet-50-style bottleneck design: a 1×1 convolution reduces/adjusts channels, a 3×3 convolution processes spatial features, and a final 1×1 convolution expands channels back up (to $4 \times$ the input channels), with a projection shortcut applied whenever spatial downsampling or channel expansion occurs.

The basic residual block uses two sequential 3×3 convolutions to learn residual features, while the bottleneck block uses a three-stage $1 \times 1 \rightarrow 3 \times 3 \rightarrow 1 \times 1$ convolution sequence to reduce, process, and expand channel dimensions trading a wider basic block for a deeper, more parameter-efficient bottleneck at the same depth.

How to phrase the distinction in your report (short but complete):

”The basic residual block uses two sequential 3×3 convolutions to learn residual features, while the bottleneck block uses a three-stage $1 \times 1 \rightarrow 3 \times 3 \rightarrow 1 \times 1$ convolution sequence to reduce, process,

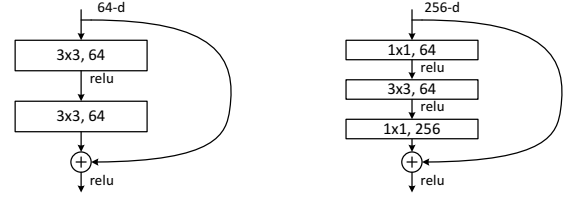


Figure 1: Structure of each block. The right one, relates to the Basic Residual block and the left one, relates to the Bottleneck Residual Block

and expand channel dimensions — trading a wider basic block for a deeper, more parameter-efficient bottleneck at the same depth.”

Structure of each block has been shown on the Figure 1.

This project, implemented on the structure of ResNet-18. This structure has 18 layers but, if this structure implements with bottleneck residual blocks, it will become ResNet-26.

3.1 Training and Inference

To train the model, all of the neurons should see the all of the data. Therefore, the forward pass splitted to two mods, train and inference.

3.1.1 Training

In the training mode, each data will pass all of the outputs. As a result, during forward pass for training, the output contains all of exit layers output. In this situation, the back prob should implement for all of the outputs independently.

3.1.2 Inference

During inference, the threshold τ will be used. After getting each exits output, the top one output will be check and it is more than the threshold, the data would be exited from the model otherwise, the data will continue the way to get out from model.

As a result of inference implementation, user should be sure about the implementation of the model. If the data is mostly hard, the exits should be mostly in the end of the model. On the other hand, if the data is mostly soft and easy-to-predict, the exits should be implemented in the first layers. The reason is related to the layer surfing of data. As it mentioned, if the top-1 answer cannot pass the threshold, then it will go to next exit. The other exits are implemented in same way. So, the data will pass and go forward until it can reach to an exit that it is in that much powerful that the output contains a top-1 answer passes the τ or in the worst case scenario, it will be wxited from the last output.

Table 2: Training Hyperparameters

Hyperparameter	Value
Optimizer	SGD
Learning rate	0.01
Momentum	0.9
Weight decay	5×10^{-4}
LR scheduler	Cosine Annealing ($T_{max} = 20$)
Epochs	20
Batch size (train/val)	64
Batch size (test)	1
Loss function	Negative Log-Likelihood (per exit)
Exit loss weighting	Equal ($w_i = 1.0$ for all exits)
Dropout rate (output head)	0.4
Validation split	0.1
Random seed	42

4 Models

The project should be implemented in three mods. ResNet without Early-Exit, with small number of Early-Exits and with a large number of Early-Exits. Therefore, this project implemented in the scale of 2 and 4 Early-Exits. The main body of all models is ResNet-18. It has 8 places to use as a output part. One, exactly after the average pooling layer and the others, exactly after each residual block.

As it mentioned before, the first Early-Exit layer is always LeNet shaped model. For the others, there is an option to customize a exit model and implement it. If the used doesn't want to implement any exit layer, the default model is a average pooling layer with one layer MLP model.

The final collection of the models are:

- Model without any Early-Exit
- Model with 2 Early-Exit
- Model with 4 Early-Exit

That should be mentioned that the final exit is mandatory otherwise our model will not have any exit.

5 Results

Here, there is results for each 3 model in 18 layers and 26 layers. Also, there is a model that only one of the exits has been train. At last, paper will report which one is in the best mode.

5.1 Normal Residual Block VS Bottleneck Residual Block

5.1.1 With equal epochs

As it is shown in figure 2, results for the normal residual network are better than bottleneck form. Accuracy for the normal residual block is something around 97%, but is it 90% for the bottleneck residual block based one.

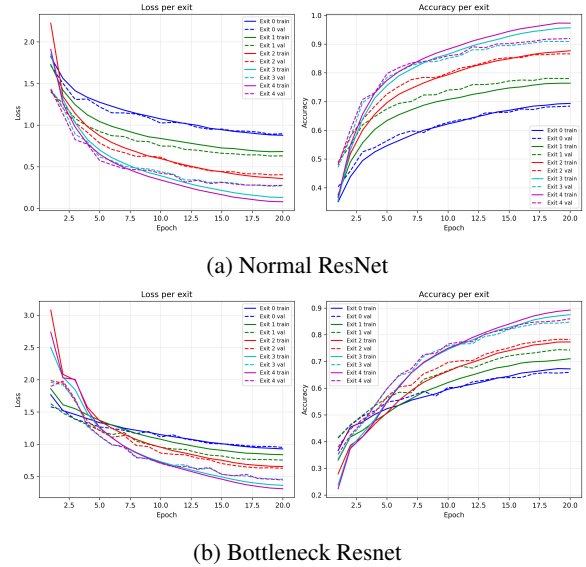


Figure 2: Per-exit training loss and accuracy over 20 epochs for (2a) the Normal ResNet and (2b) the Bottleneck ResNet, at equal training epochs.

One of the important things that should be mentioned here is that the accuracy of the first exits are lower than the last exits. This shows that how making model deeper on its way, affects the accuracy

Table 3: Early-Exit Inference Results Across Confidence Thresholds

Model	Threshold	Acc.	Time	Exit Layer Counts				Early Exit (%)
				Exit 0	Exit 1	Exit 2	Exit 3	
ResNet-18	0.1	68.40	31.85	10000	0	0	0	100.00
	0.3	69.25	31.95	9718	264	18	0	100.00
	0.5	76.32	36.66	7548	1576	658	164	99.46
	0.7	85.08	45.46	4907	2118	1647	823	94.95
	0.9	89.80	53.04	2466	1628	2435	1651	81.80
	1.0	90.67	80.90	0	0	0	0	0.00
ResNet-26	0.1	66.59	31.52	10000	0	0	0	100.00
	0.3	67.23	32.01	9698	255	40	7	100.00
	0.5	73.65	41.34	7325	1381	622	442	97.70
	0.7	80.93	50.85	4635	1596	1135	1270	86.36
	0.9	84.40	64.29	2202	1053	1305	1824	63.84
	1.0	84.67	81.58	0	0	0	0	0.00

of the model. Also, how much exits are closer to the first layers, because transforming the model to Bottleneck form affect less, their accuracy doesn't change in that much.

Other results are mentioned in the Appendix A

5.2 Test Results

For test result, there is a trade-off between accuracy and time. It has been shown in Table 3. In lower thresholds, time consumption and accuracy is lower. On the other hand, how much we increase the threshold, the accuracy and time consumption are higher. One of the best thresholds is 0.9. Around 20% time save without losing too much accuracy, only 0.2%. On ResNet-18, there is 0.8% accuracy difference but time saving is around 35%. Also, data distribution between layers are close to each other. That shows there is no resource killing or sacrificing the accuracy. Results for other models mentioned in Appendix.

5.3 Model Confidence

Accuracy is not everything. Probably model predicts wrongly but highly sure about its choice. Therefore, it is necessary to calculate the confidence of each model

As it is shown in Figure 3, the model is actually confident. Each box has a small gap that is good. Related to the (Guo et al., 2017), this is not completely confident but pretty good. This works for both Bottleneck and Normal form of ResNet.

5.4 Uni-exit Trained Model

To use model in fully functional mode, whole model should be trained. But, what will happen if only one of the exits train? There is a result for this mode. As it is completely understandable from Table 4, how deeper exit is trained, the result is more accurate in final. Also, as expected, the time consumption is worse. The important point is that, by training just one exit, accuracy is better for the times to use that specific exit compare to the time to use that specific exit in whole model training but for whole model, it makes it worse. Specially with that point that the high threshold will be used more than less threshold and in high thresholds, training whole model gives better results.

Also, unless in the situation of training just last mandatory exit, other early exits, will not be used when they do not train. In the situation of training last exit, probably because of powerful backbone, early exits can predict the data in some specific situation.

6 Discussion

There are still some challenges here. Like possibility of predicting the usage of exit before feeding inside each exit. (Kubaty et al., 2025) has implemented some methods based on this idea. But the main problem for its usage for this model is its power. The output of one convolution layer is not in that much enough to predict the usage of exit layer. The possible mistakes are letting complicated images to go inside the exit layer or directing

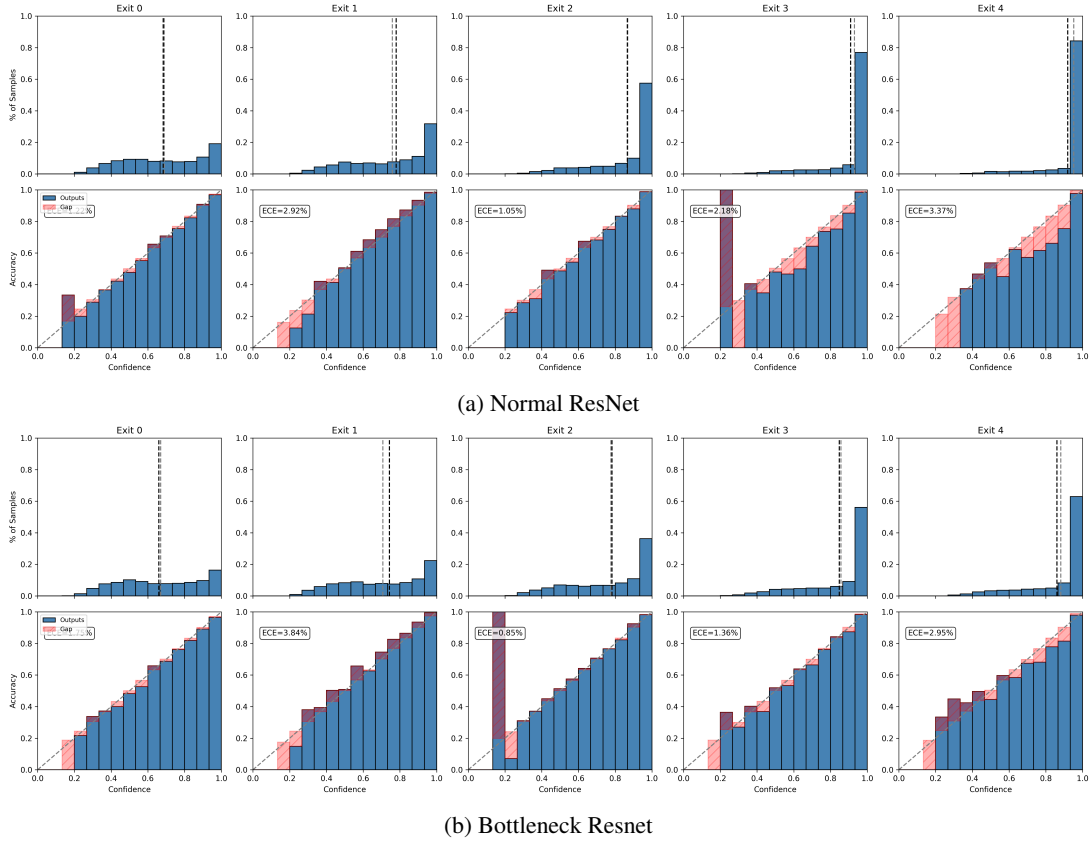


Figure 3: Per-exit confidence histograms (top) and reliability diagrams (bottom) for (3a) Normal ResNet and (3b) Bottleneck ResNet.

soft images to deeper layer. One side is killing accuracy and the other one is wasting sources.

The other challenge is calibration. How much the model is deep, it is sure about the its answer. That means the accuracy is higher but not really sure about how much it is calibrated. There is possibility of making mistake for model. Assume that something has been fed inside the model and model predict as "Cat" with possibility of 95%. Yes the model is sure about its prediction but here is not any sureness about the label. Maybe model has predicted the dog as a cat. Related to the (Guo et al., 2017) paper, there is an option to see how much is our model calibrated.

The final challenge is of the accuracy and time. This is related to the usage of the user. It can be different at time and location. Assume you are driving a car in city and want to go to the next village. You need to use roads and highways. The car is automated to predict the traffic signs. What you need is high accuracy inside the city. Because the number of traffic signs is a lot and the speed is low. So, you have enough time to predicts the signs. On the other hand, on the highways, the speed is

much higher. Less time to predict. But, there is not in that much sighs in highways like inside the city. The result is, less accuracy in shorter time.

The first scenario needs high threshold and the second one needs low threshold. We can use flexible threshold to outperform that challenge. Like as soon as the car enters the highway, the threshold drops from 0.9 to 0.5. enough to just predict the signs.

Unlike the paper worked on deeper ResNet with same layers by using bottleneck residual blocks, the results for normal ResNet is better here. One of the reasons can be related to the deepness. On the paper, they did not mentioned the method on ResNet-18. They started from ResNet-34 and transfer it to ResNet-50.

References

- Chuan Guo, Geoff Pleiss, Yu Sun, and Kilian Q. Weinberger. 2017. [On calibration of modern neural networks](#). *arXiv preprint arXiv:1706.04599*.
- Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. 2015. [Deep residual learning for image recognition](#). *arXiv preprint arXiv:1512.03385*.

Table 4: Early-Exit Inference Results Across Confidence Thresholds (Exit [0, 4, 6], ResNet-18, CIFAR-10)

Model	Threshold	Acc. (%)	Time (s)	Exit Layer Counts			Early Exit (%)
				Exit 0	Exit 1	Exit 2	
Shortcut 0	0.1	71.87	32.03	10000	0	0	100.00
	0.3	71.47	33.03	9796	0	0	97.96
	0.5	64.95	42.13	7927	0	0	79.27
	0.7	52.99	50.80	5532	0	0	55.32
	0.9	35.97	59.39	3091	0	0	30.91
	1.0	9.50	70.14	0	0	0	0.00
Shortcut 1	0.1	9.15	33.98	10000	0	0	100.00
	0.3	85.49	47.51	0	9961	0	99.61
	0.5	83.58	50.33	0	9299	0	92.99
	0.7	77.74	52.63	0	8015	0	80.15
	0.9	65.52	59.21	0	6194	0	61.94
	1.0	9.05	72.90	0	0	0	0.00
Shortcut 2	0.1	11.21	32.33	10000	0	0	100.00
	0.3	90.17	61.97	0	0	9983	99.83
	0.5	89.24	60.87	0	0	9697	96.97
	0.7	85.85	61.58	0	0	8941	89.41
	0.9	79.14	63.39	0	0	7858	78.58
	1.0	12.83	70.23	0	0	0	0.00
Shortcut 3	0.1	9.42	30.79	10000	0	0	100.00
	0.3	87.45	68.86	0	88	9	0.97
	0.5	87.89	68.66	0	36	5	0.41
	0.7	88.02	69.66	0	15	3	0.18
	0.9	88.04	68.99	0	4	3	0.07
	1.0	88.05	68.69	0	0	0	0.00

Piotr Kubaty, Filip Szatkowski, Grzegorz Choczyński, Eric Nalisnick, and Bartosz Wójcik. 2025. [Rethinking calibration for early-exit neural networks](#). *arXiv preprint arXiv:2508.21495*.

A Other Graph Results Of Models

Here is mentioned the results for remaining models. Like the first one, accuracy of the Normal ResNet model is higher than the Bottleneck model. Here should be mentioned that transforming the model to bottleneck form does not effect the first exit, therefore, accuracy of the first exit in both models are the same.

B Remaining Tables

This appendix reports two additional sets of results referenced in Section 5. Table 5 shows the threshold sweep for the 2-Early-Exit configuration,

following the same format as Table 3 in the main text. Table 5 reports the baseline results for both ResNet-18 and ResNet-26 with no early exits at all, providing the reference point against which the time and accuracy trade-offs in Tables 3 and 5 are measured.

As with the 4-Early-Exit configuration, increasing the confidence threshold trades a small amount of accuracy for a larger reduction in inference time. Compared to Table 3, the 2-Early-Exit model has fewer opportunities to exit early, so its time savings at a given threshold are [smaller/larger — worth checking the actual numbers] than the 4-Early-Exit model’s.

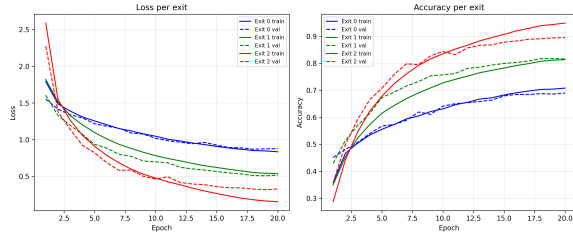
Table 6 confirms that both architectures benefit from early exiting: even the fastest early-exit configuration in Table 3 ($\tau=1.0$, which is functionally equivalent to running the full network) reports dif-

Table 5: Early-Exit Inference Results Across Confidence Thresholds (Exit Configuration [0, 4])

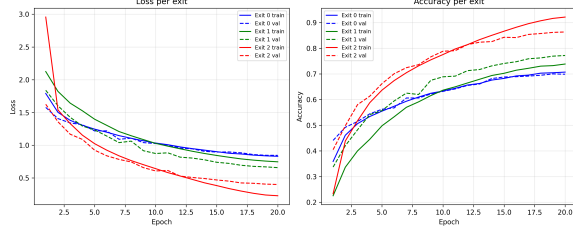
Model	Threshold	Acc. (%)	Time (s)	Exit Layer Counts		Early Exit (%)
				Exit 0	Exit 1	
ResNet-18	0.1	70.16	31.75	10000	0	100.00
	0.3	70.91	31.22	9765	223	99.88
	0.5	78.13	36.23	7748	1636	93.84
	0.7	84.78	43.34	5295	2452	77.47
	0.9	88.13	51.29	2811	2438	52.49
	1.0	88.73	63.71	0	0	0.00
ResNet-26	0.1	69.98	31.75	10000	0	100.00
	0.3	70.38	31.14	9779	196	99.75
	0.5	76.12	39.80	7778	1197	89.75
	0.7	82.28	50.05	5298	1543	68.41
	0.9	84.95	60.66	2824	1161	39.85
	1.0	85.24	76.40	0	0	0.00

Table 6: Baseline Results (No Early Exits)

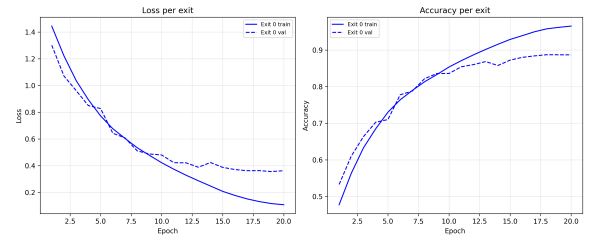
Model	Accuracy (%)	Inference Time (s)	Training Time (s)
ResNet-18	87.46	54.06	3389.62
ResNet-26	83.92	65.36	6078.84



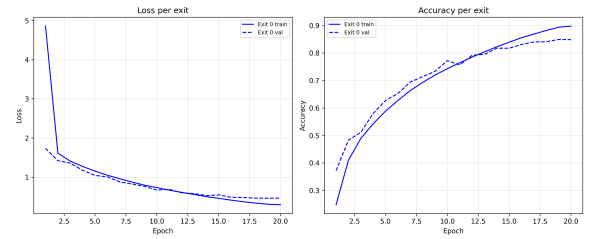
(a) Normal ResNet



(b) Bottleneck ResNet



(a) Normal ResNet



(b) Bottleneck ResNet

Figure 4: Per-exit training and validation loss (left) and accuracy (right) over 20 epochs for the 2-Early-Exit configuration, for (4a) Normal ResNet and (4b) Bottleneck ResNet. Exit 2 is the mandatory final exit; Exits 0 and 1 are the early exits.

Figure 5: Training and validation loss (left) and accuracy (right) over 20 epochs for the baseline model with no early exits, for (5a) Normal ResNet and (5b) Bottleneck ResNet.

ferent accuracy and inference time than the true baseline here, which is worth noting as a discrepancy rather than treating the two as directly comparable.

C Remaining confidences

Figures 6 and 7 show the confidence histograms and reliability diagrams for the 2-Early-Exit configuration, for the Normal and Bottleneck ResNets respectively. As with the 4-Early-Exit models, both variants show only a small gap between confidence and accuracy, though. Note here whether the gap is bigger/smaller than the 4-EE case — worth actually checking, since fewer exits could mean each one carries more ”responsibility” and might calibrate differently

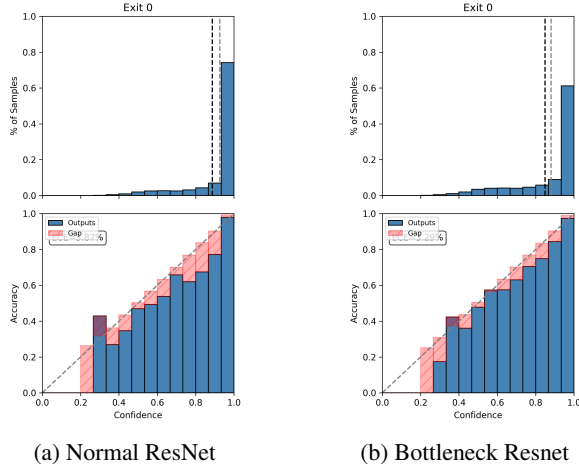


Figure 6: Confidence histograms (top) and reliability diagrams (bottom) for the baseline model, comparing (6a) Normal ResNet and (6b) Bottleneck ResNet. The reliability diagrams plot observed accuracy against predicted confidence, with the shaded region showing the calibration gap.

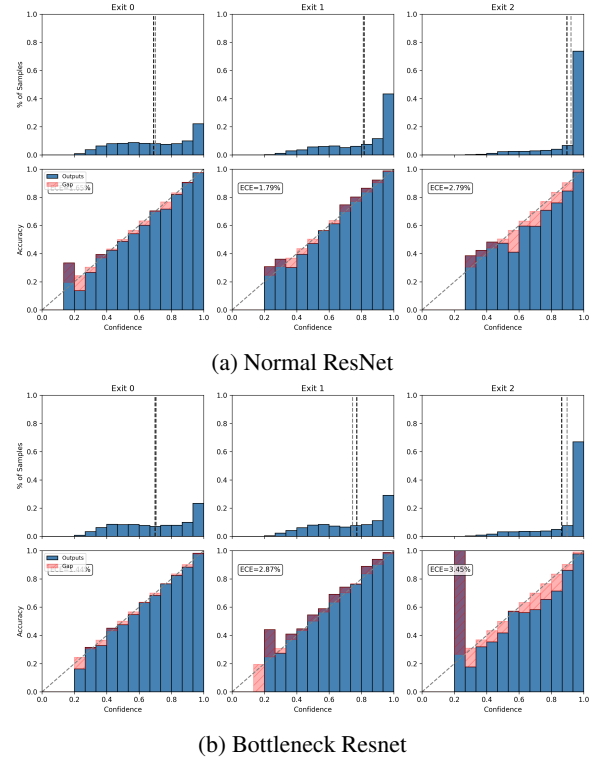


Figure 7: Confidence histograms and reliability diagrams for exits 0–2 of the 2-Early-Exit model, for (7a) Normal ResNet and (7b) Bottleneck ResNet. Expected Calibration Error (ECE) is reported for each exit; later exits show [higher/lower — check which way it actually goes] ECE than earlier ones, consistent with the accuracy-vs-calibration trade-off discussed in Section 5.3.